



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





05

Techniki IO

Metodyka

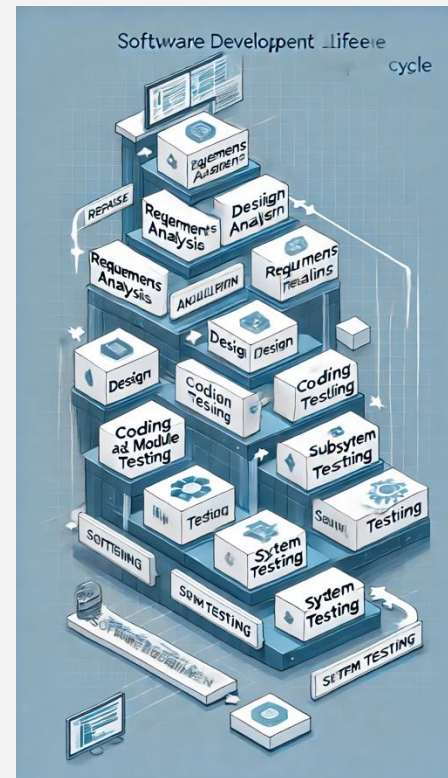


Najlepsze praktyki w działaniu!

- ✓ Notacja: język modelowania
- ✓ Techniki: sposoby postępowania
- ✓ Proces techniczny: uporządkowany proces wskazujący atomowe zadania, kolejność ich wykonania i zależności

Cykl życia oprogramowania

- 01** Analiza wymagań
- 02** Projektowanie
- 03** Kodowanie i testowanie modułów
- 04** Integracja podsystemów
- 05** Testowanie systemu



Krytyka wodospadu



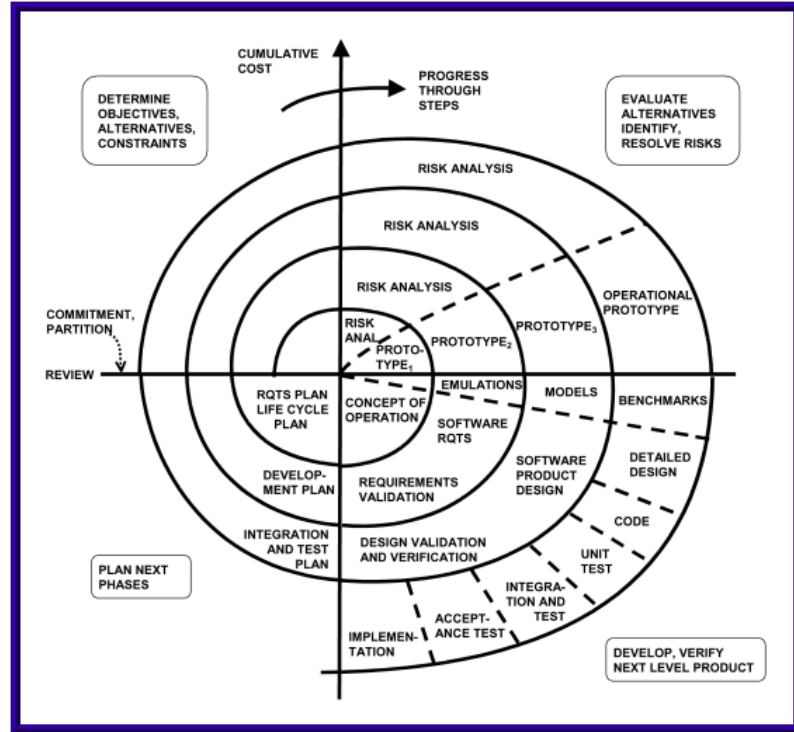
Sekwencyjnie w praktyce?

Klient określi na początku wymagania?

Użytkownik jest cierpliwy?

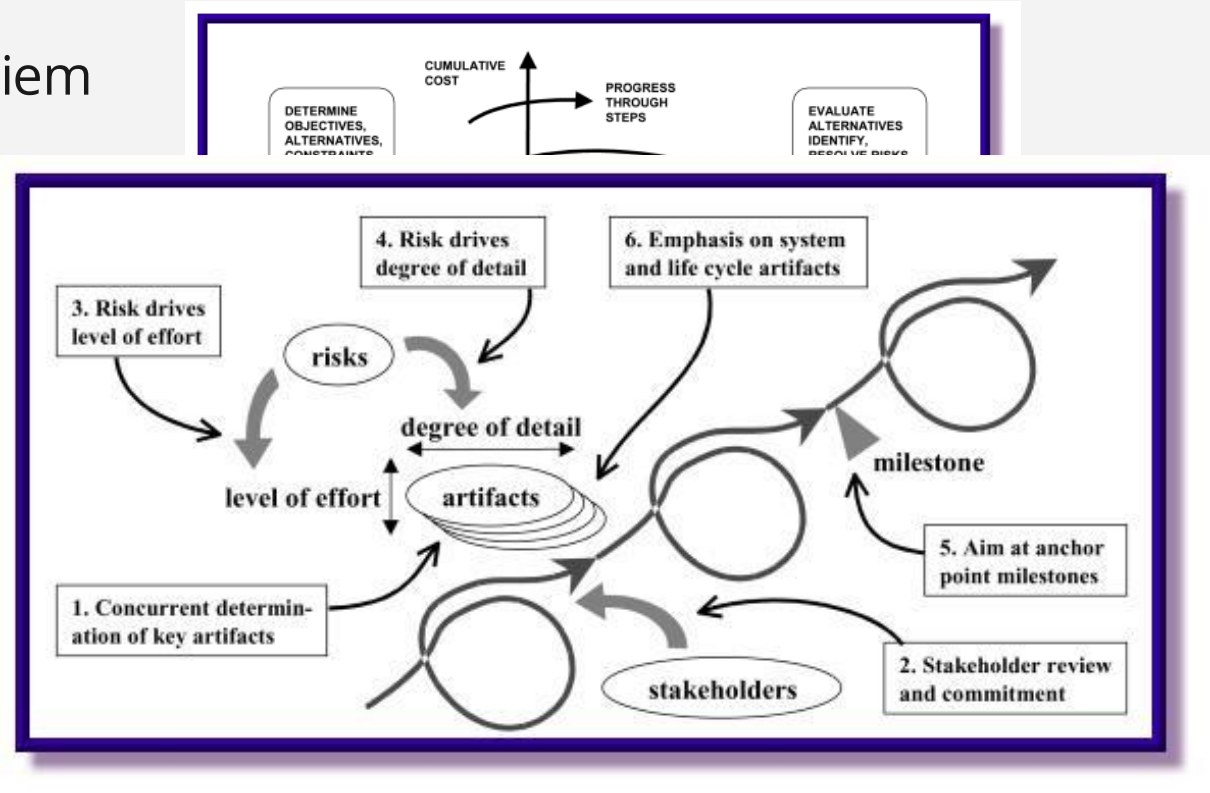
Model spiralny (Boehm 1988)

- ✓ Sterowany ryzykiem
- ✓ Cykliczny
- ✓ Punkty milowe



Model spiralny (Boehm 1988)

- ✓ Sterowany ryzykiem
- ✓ Cykliczny
- ✓ Punkty milowe



Metody formalne

Metody formalne wzbudzają wiele kontrowersji. Ich zwolennicy twierdzą, że mogą zrewolucjonizować tworzenie oprogramowania, przeciwnicy, że są zbyt skomplikowane do stosowania w praktyce. Większość wie o nich zbyt mało, żeby ocenić ich przydatność.

Antony Hall

Metody formalne



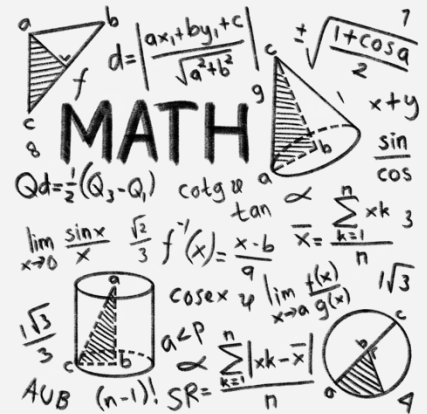
Kompletne, spójne i jednoznaczne specyfikacje



Systemy opisywane za pomocą wymagań (faktów)



Analiza specyfikacji – dowód poprawności, niesprzeczności



Metody formalne zalety



Zwięzły i dokładny opis zjawisk, obiektów i rezultatów



Płynne przejścia pomiędzy etapami tworzenia oprogramowania

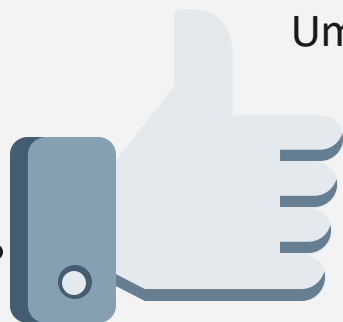


Notacja jest

Abstrakcyjna (modelowanie!)

Precyzyjna

Umożliwia weryfikację: poprawności, niesprzeczności



NIEFORMALNIE/BEZ MATEMATYKI

Sprzeczności, niejednoznaczność, niejasności, niezupełność,
różne poziomy abstrakcji

Dekalog formalistów

01 Wybierz odpowiednią notację

02 Formalizuj, ale nie przesadzaj

03 Oceniaj koszty

04 Zapewnij sobie pomoc ekspertów

05 Nie porzucaj znanych metod

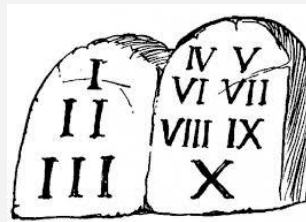
06 Dokumentuj

07 Nie rezygnuj z kontroli jakości

08 Nie bądź dogmatyczny

09 Testuj!

10 Dbaj o powtórne wykorzystanie



Metody formalne - przykłady



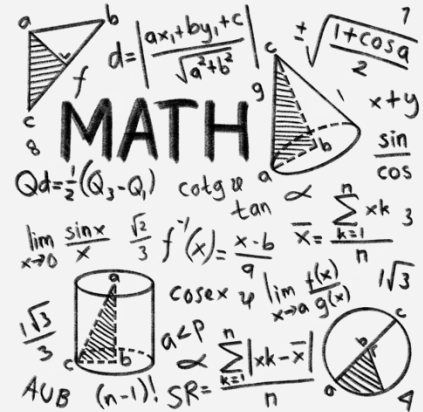
Sieci Petriego (model):
graficzny opis procesów współbieżnych oparty na
solidnych podstawach matematycznych



OCL (Object Constraint Language) (model):
opis w kontekście UML



Język Z (specyfikacje aksjomatyczne):
teoria zbiorów i logika predykatów



Metody formalne - przykłady



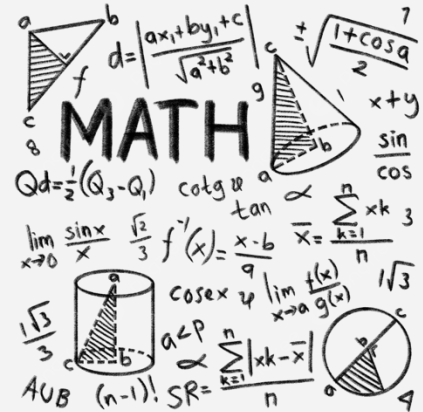
Sieci Petriego (model):
graficzny opis procesów współbieżnych oparty na
solidnych podstawach matematycznych



OCL (Object Constraint Language) (model):
opis w kontekście UML

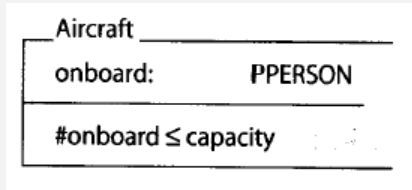


Język Z (specyfikacje aksjomatyczne):
teoria zbiorów i logika predykatów

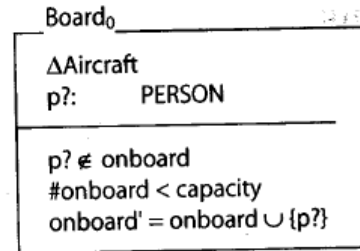


Specyfikacja Z

Samolot – stan systemu



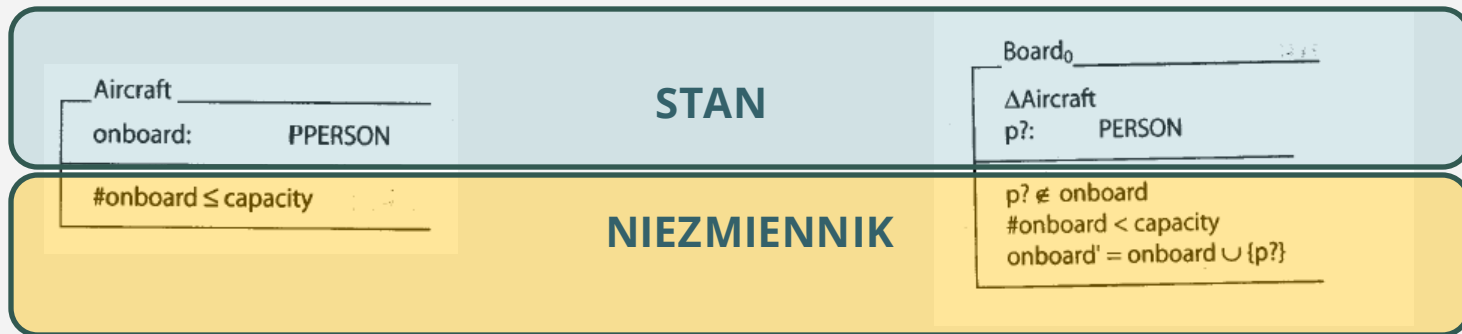
Operacja wejścia na pokład



Specyfikacja Z

Samolot – stan systemu

Operacja wejścia na pokład



Prototypowanie



Demonstracja i ustalenie wymagań



Prototyp może być wykorzystany do szkolenia użytkowników



Testowanie



Prototypowanie zalety



Wyjaśnienie nieporozumień klient-programista

**INFORMACJE OD
KLIENTA**



Ustalenie brakujących wymagań

**PRZYGOTOWANIE
PROTOTYPU**



„Działający” system na wczesnym etapie prac

**WERYFIKACJA
PROTOTYPU**



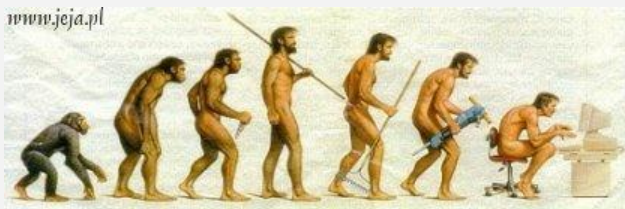
Może być punktem wyjścia do stworzenia specyfikacji wymagań



Evolutionary prototyping

CEL

Throw-away prototyping



**Dostarczenie klientowi
„działającego” systemu**

Ocena lub uzyskanie wymagań

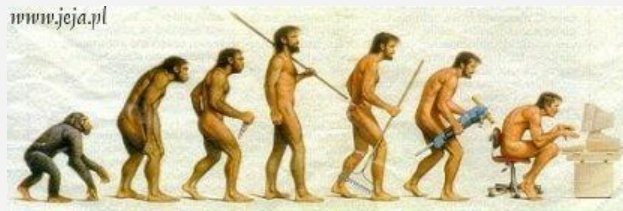
**Rozpoczynamy od najlepiej
zrozumiałych wymagań**

**Rozpoczynamy od najmniej
zrozumiałych wymagań**



Prototypowanie ewolucyjna

- ⚙️ Brak zadanej specyfikacji (systemy AI)
- ⚙️ Stosujemy techniki pozwalające na szybkie iteracje
- ⚙️ Ciągłe zmiany często skutkują poważną przebudową systemu – drogie utrzymanie
- ⚙️ Potrzebni specjaliści
- ⚙️ Taki system nie będzie „żyć” długo



Prototypowanie „do kosza”



Stosowane by zmniejszyć ryzyko błędów związanych z wymaganiami



Prototyp jest

tworzony na podstawie pierwotnej specyfikacji,
poddawany testom,
i wyrzucany do kosza



Prototyp nie może być produktem finalnym
szczętkowy
jak utrzymać?
kiepsko wykonany



Techniki prototypowania

⚙️ Języki specyfikacji formalnej

⚙️ Języki wysokiego poziomu

⚙️ Generatory aplikacji

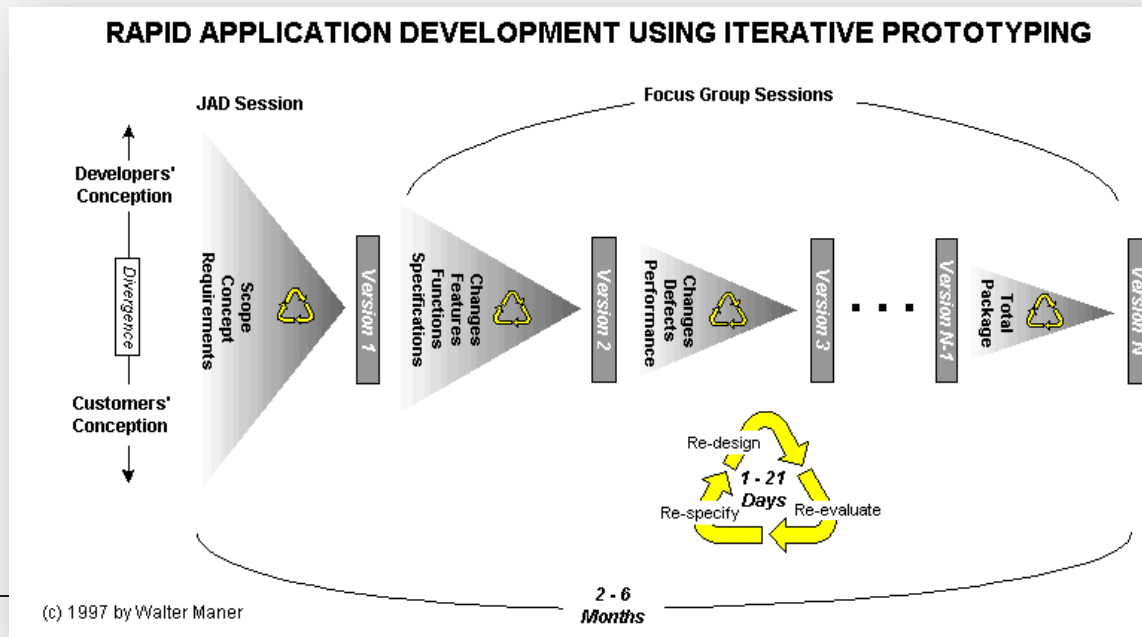
⚙️ Składanie komponentów



Rapid Application Development



Pozwala zbudować system w krótkim czasie (60-90 dni)



To RAD or not to RAD



Zakres projektu: wąski, precyzyjnie określony, jasne wymagania



Dane: istnieją i są częściowo dostępne, projekt związany z analizą danych i raportowaniem



Zespół: mały, najlepiej liczący do 6 osób



Architektura systemu: zdefiniowana i jasna, komponenty technologiczne dostępne i przetestowane



Wymagania techniczne (czas reakcji, wydajność etc.): sensowne, mniej niż 70% oficjalnie dla technologii

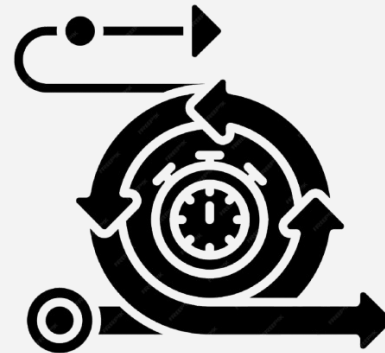
RAD czyli szybko!



Struktura procesu stawia na szybkość (listy zadań)

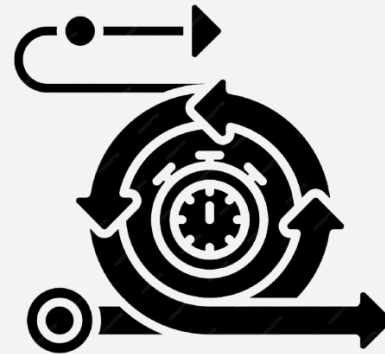


Techniki zarządzania:
prototypowanie
iteracje
timeboxing



RAD z sukcesem

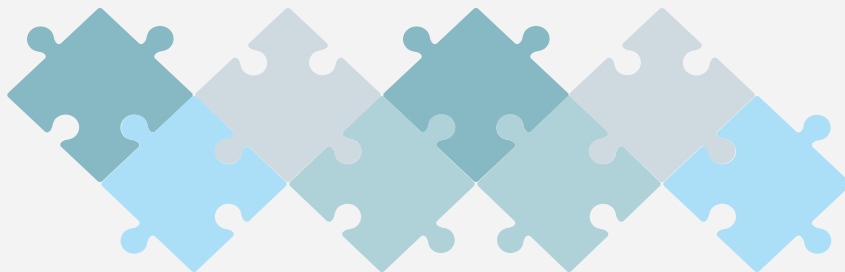
-  Niezależna aplikacja
-  Zbudowana z istniejących komponentów
-  Wydajność nie jest krytyczna
-  Wąska dystrybucja
-  Niezawodność nie jest krytyczna
-  Używana technologia co najmniej rok na rynku



Component-Based Development



Tworzenie oprogramowania z gotowych komponentów:
z różnych źródeł
napisanych w dowolnym języku
działających na różnych platformach
technologia OO



CBD: komponent



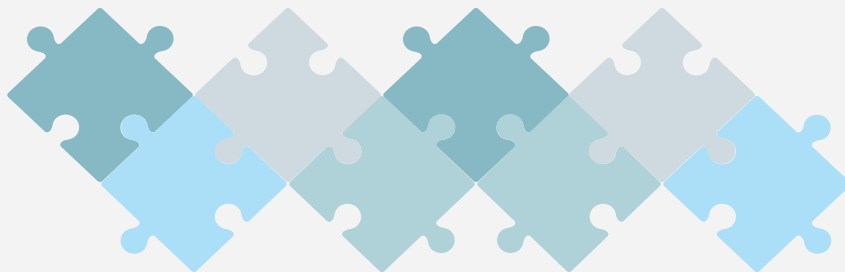
Fizyczna i zastępowalna część systemu, która realizuje zestaw interfejsów – the UML User's Guide, 1999



Czarna skrzynka posiadająca zewnętrzną specyfikację, która jest niezależna od wewnętrznych mechanizmów

Komponent:

- > Charakterystyka wewnętrzna i zewnętrzna
- > Kontekst
- > Relacje in/out



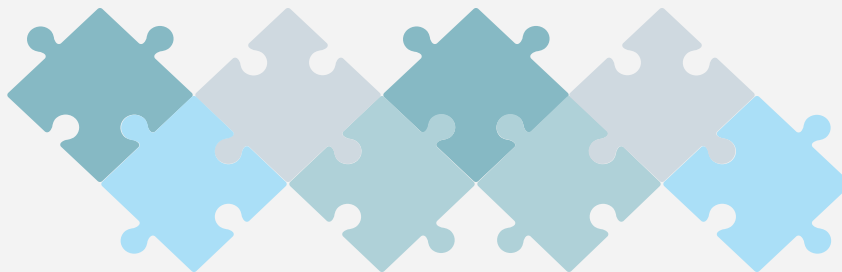
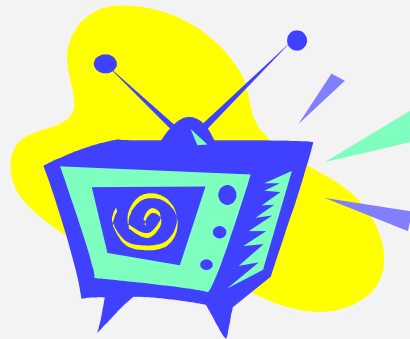
CBD: ważne terminy



Ponowne użycie



Hermetyzacja i korzystanie z interfejsów



CBD: zalety



Mniejszy koszt



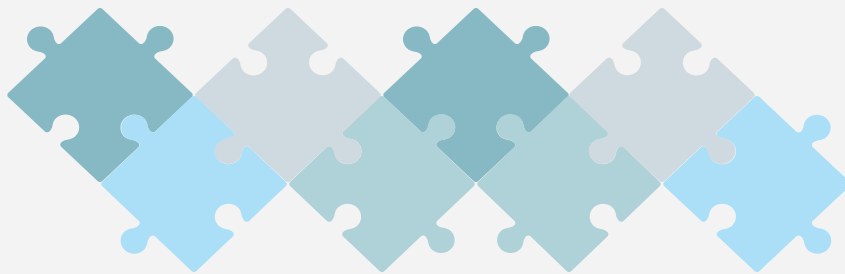
Im częściej używany komponent, tym cenniejszy



Szybka i niekosztowna personalizacja



Mniej błędów



Concurrent Development

⚙️ Szybciej, szybciej, szybciej!

⚙️ Przejście z modelu sekwencyjnego do współbieżnego

⚙️ Redukcja czasu kosztem zwiększenia złożoności procesu
Stąd częste porażki 😞



Concurrent Development



Większa częstotliwość i liczba wymian informacji pomiędzy fazami projektowymi



Więcej zadań rozpoczyna się z niekompletnymi lub wstępnymi wymaganiami



Sekwencyjne zarządzanie zawodzi



Concurrent Development

CZAS BUDOWY	30% - 70% KRÓTSZY
CZAS WDROŻENIA	20% - 90% KRÓTSZY
OGÓLNA JAKOŚĆ	200% - 600% WYŻSZA
ZYSKI	5% - 50% WIĘKSZE
ZWROT KOSZTÓW	20% - 120% WYŻSZY
WYDAJNOŚĆ EKSPERTÓW	20% - 110% WYŻSZA



National Institute of Standards & Technology, Thomas Group Inc. Published in Business Week Special Report, 30.IV.1990

Aspect Oriented Software Development



<https://aosd.net>



Aspect Oriented Programming: Gregor Kiczales, Xerox PARC '97



Dekompozycja problemu



FUNKCJONALNA: obiekty, moduły, procedury, itd.



ASPEKTOWA: dotyczy kilku elementów funkcjonalnych

synchronizacja, komunikacja

nie da się tego „czysto” zrobić stosując takie techniki jak OO



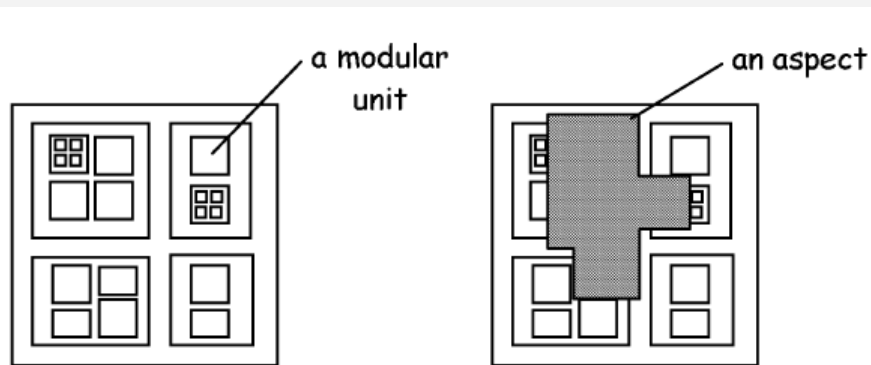
Dekompozycja problemu



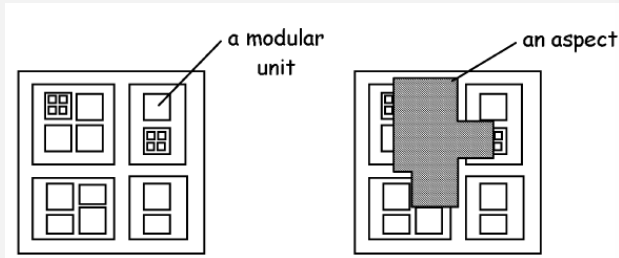
FUNKCJONALNA: obiekty, moduły, procedury, itd.



ASPEKTOWA: dotyczy kilku elementów funkcjonalnych
synchronizacja, komunikacja
nie da się tego „czysto” zrobić stosując takie techniki jak OO

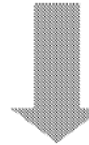


Dekompozycja problemu



```
Module "Aspect" 10 line 1
  10 line 2
  10 line 3
  10 line 4
  10 line 5
  10 line 6
  10 line 7
  10 line 8
  10 line 9
  10 line 10
  10 line 11
  10 line 12
  10 line 13
  10 line 14
  10 line 15
  10 line 16
  10 line 17
  10 line 18
  10 line 19
  10 line 20
  10 line 21
  10 line 22
  10 line 23
  10 line 24
  10 line 25
  10 line 26
  10 line 27
  10 line 28
  10 line 29
  10 line 30
  10 line 31
  10 line 32
  10 line 33
  10 line 34
  10 line 35
  10 line 36
  10 line 37
  10 line 38
  10 line 39
  10 line 40
  10 line 41
  10 line 42
  10 line 43
  10 line 44
  10 line 45
  10 line 46
  10 line 47
  10 line 48
  10 line 49
  10 line 50
  10 line 51
  10 line 52
  10 line 53
  10 line 54
  10 line 55
  10 line 56
  10 line 57
  10 line 58
  10 line 59
  10 line 60
  10 line 61
  10 line 62
  10 line 63
  10 line 64
  10 line 65
  10 line 66
  10 line 67
  10 line 68
  10 line 69
  10 line 70
  10 line 71
  10 line 72
  10 line 73
  10 line 74
  10 line 75
  10 line 76
  10 line 77
  10 line 78
  10 line 79
  10 line 80
  10 line 81
  10 line 82
  10 line 83
  10 line 84
  10 line 85
  10 line 86
  10 line 87
  10 line 88
  10 line 89
  10 line 90
  10 line 91
  10 line 92
  10 line 93
  10 line 94
  10 line 95
  10 line 96
  10 line 97
  10 line 98
  10 line 99
  10 line 100
```

code with
merged
crosscutting
structures



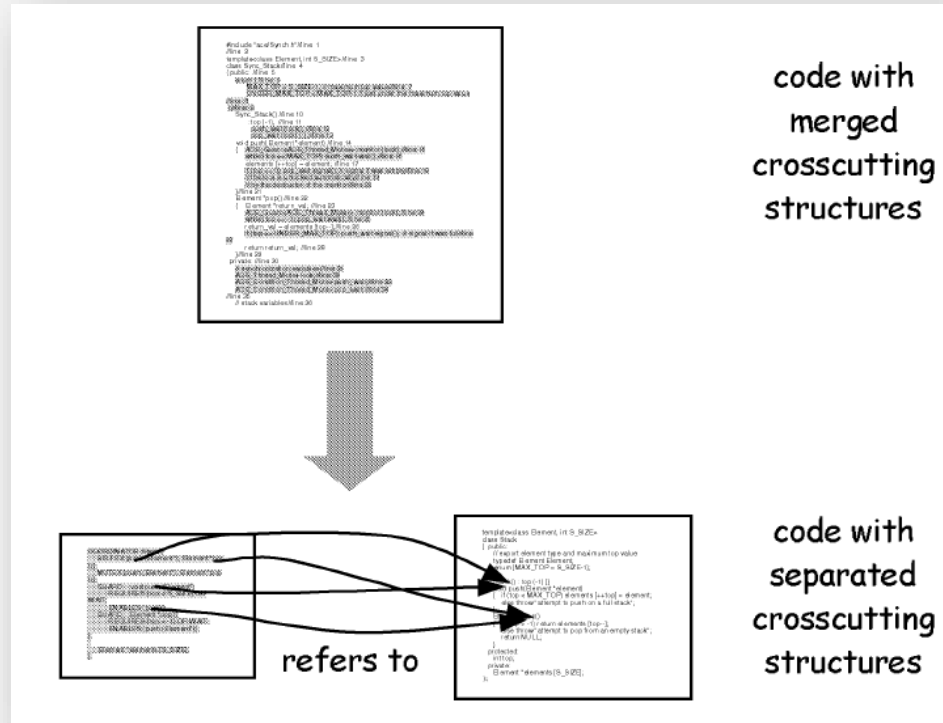
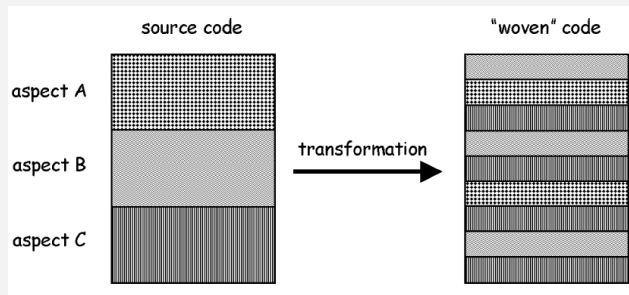
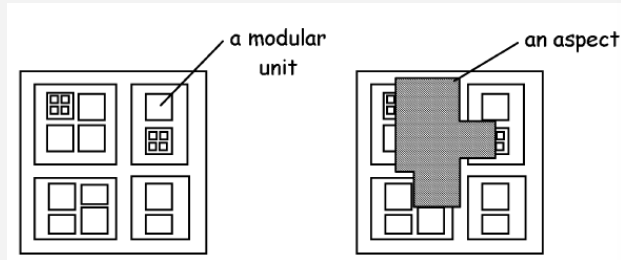
```
Module "Aspect" 10 line 1
  10 line 2
  10 line 3
  10 line 4
  10 line 5
  10 line 6
  10 line 7
  10 line 8
  10 line 9
  10 line 10
  10 line 11
  10 line 12
  10 line 13
  10 line 14
  10 line 15
  10 line 16
  10 line 17
  10 line 18
  10 line 19
  10 line 20
  10 line 21
  10 line 22
  10 line 23
  10 line 24
  10 line 25
  10 line 26
  10 line 27
  10 line 28
  10 line 29
  10 line 30
  10 line 31
  10 line 32
  10 line 33
  10 line 34
  10 line 35
  10 line 36
  10 line 37
  10 line 38
  10 line 39
  10 line 40
  10 line 41
  10 line 42
  10 line 43
  10 line 44
  10 line 45
  10 line 46
  10 line 47
  10 line 48
  10 line 49
  10 line 50
  10 line 51
  10 line 52
  10 line 53
  10 line 54
  10 line 55
  10 line 56
  10 line 57
  10 line 58
  10 line 59
  10 line 60
  10 line 61
  10 line 62
  10 line 63
  10 line 64
  10 line 65
  10 line 66
  10 line 67
  10 line 68
  10 line 69
  10 line 70
  10 line 71
  10 line 72
  10 line 73
  10 line 74
  10 line 75
  10 line 76
  10 line 77
  10 line 78
  10 line 79
  10 line 80
  10 line 81
  10 line 82
  10 line 83
  10 line 84
  10 line 85
  10 line 86
  10 line 87
  10 line 88
  10 line 89
  10 line 90
  10 line 91
  10 line 92
  10 line 93
  10 line 94
  10 line 95
  10 line 96
  10 line 97
  10 line 98
  10 line 99
  10 line 100
```

refers to

```
Module "Aspect" 10 line 1
  10 line 2
  10 line 3
  10 line 4
  10 line 5
  10 line 6
  10 line 7
  10 line 8
  10 line 9
  10 line 10
  10 line 11
  10 line 12
  10 line 13
  10 line 14
  10 line 15
  10 line 16
  10 line 17
  10 line 18
  10 line 19
  10 line 20
  10 line 21
  10 line 22
  10 line 23
  10 line 24
  10 line 25
  10 line 26
  10 line 27
  10 line 28
  10 line 29
  10 line 30
  10 line 31
  10 line 32
  10 line 33
  10 line 34
  10 line 35
  10 line 36
  10 line 37
  10 line 38
  10 line 39
  10 line 40
  10 line 41
  10 line 42
  10 line 43
  10 line 44
  10 line 45
  10 line 46
  10 line 47
  10 line 48
  10 line 49
  10 line 50
  10 line 51
  10 line 52
  10 line 53
  10 line 54
  10 line 55
  10 line 56
  10 line 57
  10 line 58
  10 line 59
  10 line 60
  10 line 61
  10 line 62
  10 line 63
  10 line 64
  10 line 65
  10 line 66
  10 line 67
  10 line 68
  10 line 69
  10 line 70
  10 line 71
  10 line 72
  10 line 73
  10 line 74
  10 line 75
  10 line 76
  10 line 77
  10 line 78
  10 line 79
  10 line 80
  10 line 81
  10 line 82
  10 line 83
  10 line 84
  10 line 85
  10 line 86
  10 line 87
  10 line 88
  10 line 89
  10 line 90
  10 line 91
  10 line 92
  10 line 93
  10 line 94
  10 line 95
  10 line 96
  10 line 97
  10 line 98
  10 line 99
  10 line 100
```

code with
separated
crosscutting
structures

Dekompozycja problemu



Dekompozycja problemu

```
/**  
 * not a dog in sight...  
 */
```

```
public class Person {  
  
    private String lastName;  
    private Address address;  
    ...  
  
    public String getLastName() { return lastName; }  
  
    ...  
}
```

```
/**  
 * not a person in sight...  
 */
```

```
public aspect DogOwnership {  
  
    public interface IDogOwner {};  
  
    private int IDogOwner.numDogsOwned;  
  
    public int IDogOwner.getNumDogsOwned() { return numDogsOwned; }  
  
}
```

```
public aspect PersonOwnsDog {  
    declare parents : Person implements DogOwnership.IDogOwner;  
}
```

```
Person person = new Person();  
person.getNumDogsOwned();
```

```
...  
if (person instanceof DogOwnership.IDogOwner) {  
    ((DogOwnership.IDogOwner) person).getNumDogsOwned();  
}  
...
```

Cel AOSD



Dostarczenie metod i technik umożliwiających funkcjonalną i aspektową dekompozycję problemu



Aspekty „przecinają” komponenty funkcjonalne



Dostarczenie metod i technik umożliwiających złożenie komponentów funkcjonalnych i aspektów w spójną implementację systemu



AOSD: przykładowe aspekty



SYSTEMY ROZPROSZONE

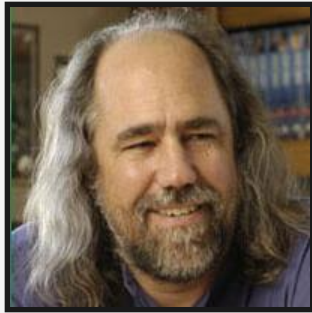
SYNCHRONIZACJA	SYNCHRONIZACJA
KONTROLA BŁĘDÓW	ZDALNE WYWOŁANIE
ZARZĄDZANIE PAMIĘCIĄ	STRATEGIE PRZEKAZYWANIA PARAMETRÓW
BEZPIECZEŃSTWO	REPLIKACJA
TESTOWANIE	OBSŁUGA WYJĄTKÓW
REPREZENTACJA DANYCH	ROZPROSZONE TRANSAKCJE

IMPLEMENTACJE

- ✓ C#/VB.NET: Puzzle.Naspect, AspectDNG, Aspect# ...
- ✓ Java: AspectJ, Jakarta Hivemind, JBoss AOP, InjectJ ...
- ✓ C/C++: AspectC++, XWeaver project, AspectC ...
- ✓ Python, PHP, JavaScript, Perl, Common Lisp, XML, ColdFusion, inne..

The Rational Unified Process

1995 Rational Software "three Amigos"



Grady Booch



Ivar Hjalmar Jacobson



James Rumbaugh

The Rational Unified Process



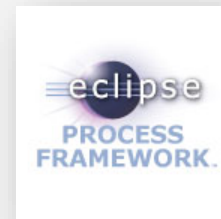
2003 IBM przejmuje Rationala



RUP → IBM Rational Method Composer



Eclipse Foundation: Eclipse Process Framework Project (ostatnie wydanie 2018)





The Spirit of the RUP



Od początku i na bieżąco rozwiązuj najbardziej ryzykowne kwestie



Lista największych zagrożeń



Koncentruj się na działającym oprogramowaniu



Plany i projektowanie jest dobre – działające oprogramowanie jest lepsze



Realizuj tylko niezbędne artefakty



The Spirit of the RUP



Koncentruj się na potrzebach klienta



Iteracje



Przypadki użycia

Projektowanie, implementacja i testowanie skoncentrowane na
potrzebach klienta



Dokumentowanie wymagań jest dobre –realizacja lepsza



The Spirit of the RUP



Szybko uwzględniaj zmiany
Procedury wprowadzania zmian
Ocena konsekwencji danej zmiany
Minimalizacja kosztów zmiany



Zaczynij od architektury
Najpierw zbuduj szkielet, potem uzupełnij brakujące elementy



Buduj z komponentów



The Spirit of the RUP



Stwórz zespół

Nie buduj zespołów wokół pojedynczych funkcji
Rezultaty są współwłasnością członków zespołu



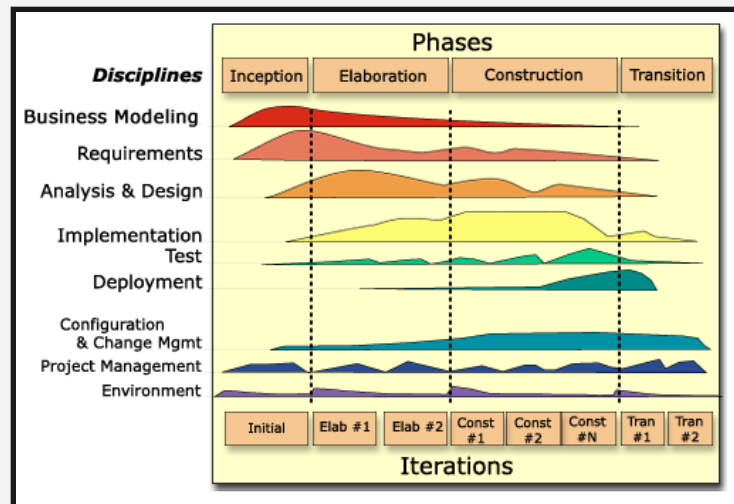
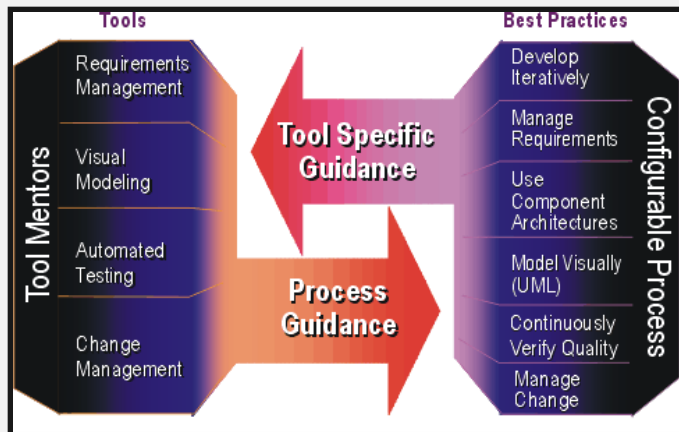
Zapewnienie jakości determinuje realizację

Quality By Design

Testowanie od początku realizacji



Podstawa RUP



Co daje nam RUP?

- 01 Pomaga poprawiać jakość
- 02 Pomaga zmieścić się w czasie i budżecie
- 03 Nie zależy od poświęcenia jednostek
- 04 Optimalizacja procesu
- 05 Przewidywalność i powtarzalność
- 06 Koncentracja na produkcie
- 07 Efektywna adaptacja nowych technik i narzędzi





Koniec!